



INSTITUTO POLITÉCNICO NACIONAL



INSTITUTO POLITÉCNICO NACIONAL

ESCUELA SUPERIOR DE CÓMPUTO

SISTEMAS EN CHIP

Reporte de la práctica 16

“Termómetro de 0 °C a 50 °C (32 °F a 122 °F)”

Grupo: 6CM1

Integrantes

- García Escamilla Bryan Alexis
- Meléndez Macedonio Rodrigo

Profesor

Aguilar Sánchez Fernando

Fecha de entrega: 18 de marzo de 2025

INTRODUCCIÓN

Termómetro

En el ámbito de la electrónica, un termómetro no es solo un dispositivo de visualización, sino un sistema completo de adquisición de datos que traduce la agitación térmica de las moléculas en una señal eléctrica procesable.

A diferencia de los termómetros de mercurio, los electrónicos destacan por su precisión, velocidad de respuesta y capacidad de integrarse a sistemas de control.

Sensores de Temperatura: El Corazón del Sistema

Existen cuatro tecnologías principales para detectar el calor, cada una basada en un principio físico distinto:

A. Termistores (NTC y PTC)

Son resistencias que cambian su valor drásticamente con la temperatura.

- **NTC (Negative Temperature Coefficient):** Su resistencia baja cuando la temperatura sube. Son los más comunes por su alta sensibilidad en rangos de -50 a 150 °C.
- **PTC (Positive Temperature Coefficient):** Su resistencia sube con la temperatura. Se usan a menudo como fusibles térmicos de protección.

RTD (Resistance Temperature Detectors)

Suelen estar hechos de metales puros como el Platino (ej. Pt100).

- **Principio:** La resistencia eléctrica de un metal aumenta de forma muy lineal con la temperatura.
- **Uso:** Son los estándares de la industria cuando se requiere una precisión extrema y estabilidad a largo plazo.

Termopares (Thermocouples)

Basados en el Efecto Seebeck. Consisten en la unión de dos metales distintos que generan un voltaje muy pequeño (mV) proporcional a la diferencia de temperatura entre la unión y los terminales.

- **Uso:** Ideales para temperaturas extremas (desde hornos industriales hasta criogenia).

Sensores de Circuito Integrado (IC)

Son chips que entregan una salida ya acondicionada, ya sea un voltaje lineal (como el LM35, que da 10 mV por cada grado Celsius) o una señal digital (como el DS18B20 vía protocolo 1-Wire).

Acondicionamiento de la Señal

La señal de un sensor rara vez es perfecta para un procesador. Se requiere un circuito intermedio:

1. **Divisor de Voltaje o Puente de Wheatstone:** Para convertir el cambio de resistencia (de un NTC o RTD) en un cambio de voltaje.
2. **Amplificación:** Especialmente para termopares, cuya señal es tan débil que necesita amplificadores de instrumentación.
3. **Filtrado:** Para eliminar el ruido eléctrico (como el zumbido de 60 Hz de las líneas eléctricas) que puede falsear la lectura.

Conversión y Visualización

Una vez que el voltaje está limpio, el proceso sigue el flujo digital estándar:

- **ADC:** El voltaje analógico se convierte en un número digital.
- **Linealización:** Mediante fórmulas matemáticas (como la ecuación de *Steinhart-Hart* para termistores), el microcontrolador convierte ese número en grados Celsius o Fahrenheit.
- **Salida:** El resultado se muestra en un LCD o display de 7 segmentos.

DESARROLLO

Circuito en hecho en Proteus

Para el desarrollo de la práctica 16, el circuito a armar es el siguiente:

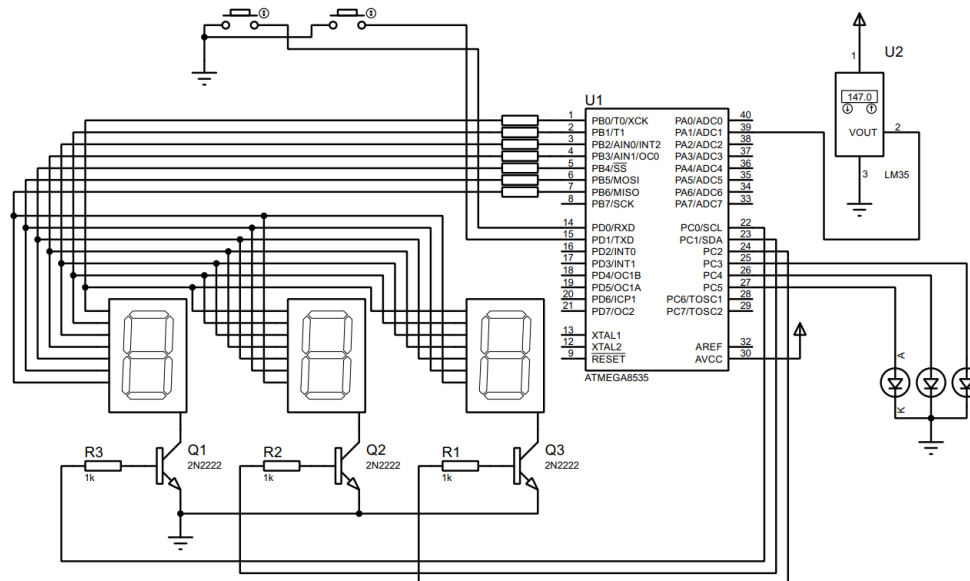


Imagen 1. Circuito creado en Proteus.

Configuración previa al código

De acuerdo con la imagen, el registro B se configura como de salida (PB0-PB6) al igual que el registro C (PC0-PC5), mientras que el registro D como de entrada (PD0 y PD1) activando las resistencias de pull-up.

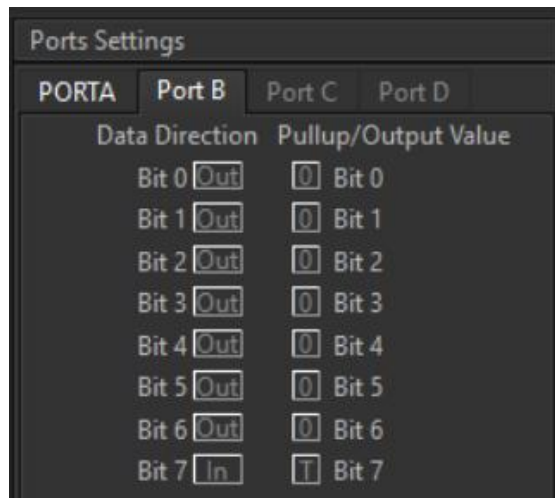


Imagen 2. Configuración del puerto B.

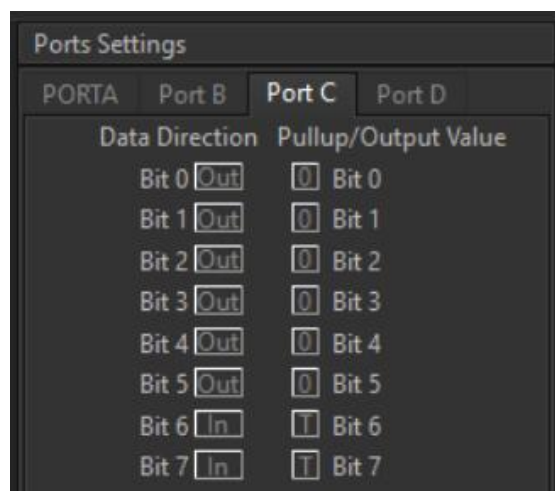


Imagen 3. Configuración del puerto C.

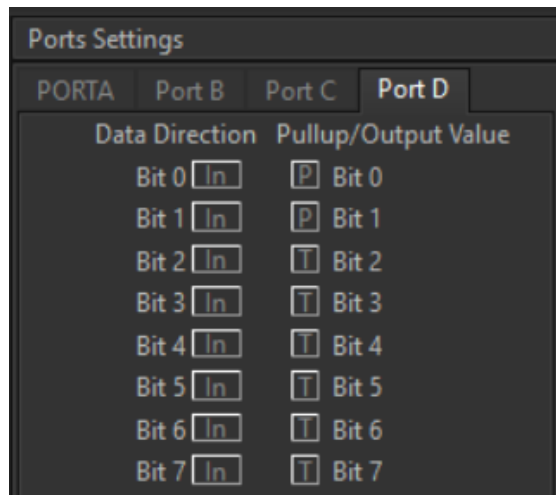


Imagen 4. Configuración del puerto D.

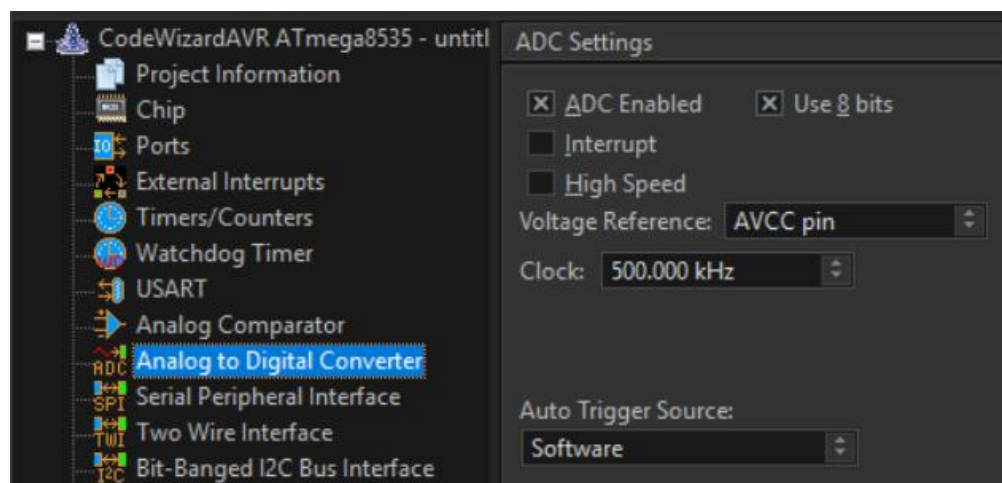


Imagen 5. Configuración del convertidor analógico digital.

Código del circuito de CodeVision:

```
1  /*****
2  This program was created by the CodeWizardAVR V4.06a
3  Automatic Program Generator
4  Copyright 1998-2025 Pavel Haiduc, HP InfoTech S.R.L.
5  http://www.hpinfotech.ro
6
7  Project : Pr  ctica 16 'Term  metro    a 50   C (32 a 122   F)'
8  Version : 1.0
9  Date    : 13/03/2026
10 Author  : Garc  a Escamilla Bryan Alexis
11 Company :
12 Comments:
13
14
15 Chip type           : ATmega8535
16 Program type        : Application
17 AVR Core Clock frequency: 1.000000 MHz
18 Memory model        : Small
19 External RAM size    : 0
20 Data Stack size     : 128
21 *****/
22
23 // I/O Registers definitions
24 #include <mega8535.h>
25
26 // Delay functions
27 #include <delay.h>
```

```
29 // Declare your global variables here
30 unsigned char cn, intensidad;
31 unsigned char unidades, decenas, centenas;
32 bit isFahrenheit = 0;
33 bit pasadoCF = 1, actualCF;
34 bit pasadoMODE = 1, actualMODE;
35 const char tabla7segmentos[10] = {0x3F, 0x06, 0x5b, 0x4f, 0x66, 0x6d, 0x7d, 0x07, 0x7f, 0x6f};
36
37 unsigned char modo = 0;           // 0:Real-time, 1:Max, 2:Min
38 unsigned int max_temp = 0;
39 unsigned int min_temp = 0xFFFF;
40
41
42 // ADC Voltage Reference: AVCC pin
43 #define ADC_VREF_TYPE ((0<<REFS1) | (1<<REFS0) | (1<<ADLAR))
44 #define LED_RED PORTC.3
45 #define LED_GREEN PORTC.4
46 #define LED_BLUE PORTC.5
47 #define BTN_CF PIND.0
48 #define BTN_MODE PIND.1
49 #define C0 PORTC.0
50 #define C1 PORTC.1
51 #define C2 PORTC.2
```

```

53 // Read the 8 most significant bits
54 // of the AD conversion result
55 // Read Voltage=read_adc*(Vref/256.0)
56 unsigned char read_adc(unsigned char adc_input) {
57     ADMUX=adc_input | ADC_VREF_TYPE;
58     // Delay needed for the stabilization of the ADC input voltage
59     delay_us(10);
60     // Start the AD conversion
61     ADCSRA|=(1<<ADSC);
62     // Wait for the AD conversion to complete
63     while ((ADCSRA & (1<<ADIF))==0);
64     ADCSRA|=(1<<ADIF);
65     return ADCH;
66 }

```

```

68 void main(void) {
69     // Declare your Local variables here
70
71     // Input/Output Ports initialization
72     // Port A initialization
73     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
74     DDRA=(0<<DDA7) | (0<<DDA6) | (0<<DDA5) | (0<<DDA4) | (0<<DDA3) | (0<<DDA2) | (0<<DDA1) | (0<<DDA0);
75     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=T Bit0=T
76     PORTA=(0<<PORTA7) | (0<<PORTA6) | (0<<PORTA5) | (0<<PORTA4) | (0<<PORTA3) | (0<<PORTA2) | (0<<PORTA1) | (0<<PORTA0);
77
78     // Port B initialization
79     // Function: Bit7=In Bit6=Out Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
80     DDRB=(0<<ddb7) | (1<<ddb6) | (1<<ddb5) | (1<<ddb4) | (1<<ddb3) | (1<<ddb2) | (1<<ddb1) | (1<<ddb0);
81     // State: Bit7=T Bit6=0 Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
82     PORTB=(0<<PORTB7) | (0<<PORTB6) | (0<<PORTB5) | (0<<PORTB4) | (0<<PORTB3) | (0<<PORTB2) | (0<<PORTB1) | (0<<PORTB0);
83
84     // Port C initialization
85     // Function: Bit7=In Bit6=In Bit5=Out Bit4=Out Bit3=Out Bit2=Out Bit1=Out Bit0=Out
86     DDRC=(0<<ddc7) | (0<<ddc6) | (1<<ddc5) | (1<<ddc4) | (1<<ddc3) | (1<<ddc2) | (1<<ddc1) | (1<<ddc0);
87     // State: Bit7=T Bit6=T Bit5=0 Bit4=0 Bit3=0 Bit2=0 Bit1=0 Bit0=0
88     PORTC=(0<<PORTC7) | (0<<PORTC6) | (0<<PORTC5) | (0<<PORTC4) | (0<<PORTC3) | (0<<PORTC2) | (0<<PORTC1) | (0<<PORTC0);
89
90     // Port D initialization
91     // Function: Bit7=In Bit6=In Bit5=In Bit4=In Bit3=In Bit2=In Bit1=In Bit0=In
92     DDRD=(0<<ddd7) | (0<<ddd6) | (0<<ddd5) | (0<<ddd4) | (0<<ddd3) | (0<<ddd2) | (0<<ddd1) | (0<<ddd0);
93     // State: Bit7=T Bit6=T Bit5=T Bit4=T Bit3=T Bit2=T Bit1=P Bit0=P
94     PORTD=(0<<PORTD7) | (0<<PORTD6) | (0<<PORTD5) | (0<<PORTD4) | (0<<PORTD3) | (0<<PORTD2) | (1<<PORTD1) | (1<<PORTD0);

```

```

96 // Timer/Counter 0 initialization
97 // Clock source: System Clock
98 // Clock value: Timer 0 Stopped
99 // Mode: Normal top=0xFF
100 // OC0 output: Disconnected
101 TCCR0=(0<<WGM00) | (0<<WGM01) | (0<<COM00) | (0<<COM01) | (0<<CS02) | (0<<CS01) | (0<<CS00);
102 TCNT0=0x00;
103 OCR0=0x00;

```



```

105 // Timer/Counter 1 initialization
106 // Clock source: System Clock
107 // Clock value: Timer1 Stopped
108 // Mode: Normal top=0xFFFF
109 // OC1A output: Disconnected
110 // OC1B output: Disconnected
111 // Noise Canceler: Off
112 // Input Capture on Falling Edge
113 // Timer1 Overflow Interrupt: Off
114 // Input Capture Interrupt: Off
115 // Compare A Match Interrupt: Off
116 // Compare B Match Interrupt: Off
117 TCCR1A=(0<<COM1A1) | (0<<COM1A0) | (0<<COM1B1) | (0<<COM1B0) | (0<<WGM11) | (0<<WGM10);
118 TCCR1B=(0<<ICNC1) | (0<<ICES1) | (0<<WGM13) | (0<<WGM12) | (0<<CS12) | (0<<CS11) | (0<<CS10);
119 TCNT1H=0x00;
120 TCNT1L=0x00;
121 ICR1H=0x00;
122 ICR1L=0x00;
123 OCR1AH=0x00;
124 OCR1AL=0x00;
125 OCR1BH=0x00;
126 OCR1BL=0x00;

```

```

128 // Timer/Counter 2 initialization
129 // Clock source: System Clock
130 // Clock value: Timer2 Stopped
131 // Mode: Normal top=0xFF
132 // OC2 output: Disconnected
133 ASSR=0<<AS2;
134 TCCR2=(0<<WGM20) | (0<<COM21) | (0<<COM20) | (0<<WGM21) | (0<<CS22) | (0<<CS21) | (0<<CS20);
135 TCNT2=0x00;
136 OCR2=0x00;
137
138 // Timer(s)/Counter(s) Interrupt(s) initialization
139 TIMSK=(0<<OCIE2) | (0<<TOIE2) | (0<<TICIE1) | (0<<OCIE1A) | (0<<OCIE1B) | (0<<TOIE1) | (0<<OCIE0) | (0<<TOIE0);
140
141 // External Interrupt(s) initialization
142 // INT0: Off
143 // INT1: Off
144 // INT2: Off
145 MCUCR=(0<<ISC11) | (0<<ISC10) | (0<<ISC01) | (0<<ISC00);
146 MCUCSR=(0<<ISC2);
147
148 // USART initialization
149 // USART disabled
150 UCSRB=(0<<RXCIE) | (0<<TXCIE) | (0<<UDRIE) | (0<<RXEN) | (0<<TXEN) | (0<<UCSZ2) | (0<<RXB8) | (0<<TXB8);

```



```

152 // Analog Comparator initialization
153 // Analog Comparator: Off
154 // The Analog Comparator's positive input is
155 // connected to the AIN0 pin
156 // The Analog Comparator's negative input is
157 // connected to the AIN1 pin
158 ACSR=(1<<ACD) | (0<<ACBG) | (0<<ACO) | (0<<ACI) | (0<<ACIE) | (0<<ACIC) | (0<<ACIS1) | (0<<ACIS0);
159
160 // ADC initialization
161 // ADC Clock frequency: 500.000 kHz
162 // ADC High Speed Mode: Off
163 // ADC Auto Trigger Source: Software
164 // Only the 8 most significant bits of
165 // the AD conversion result are used
166 ADMUX=ADC_VREF_TYPE;
167 ADCSRA=(1<<ADEN) | (0<<ADSC) | (0<<ADATE) | (0<<ADIF) | (0<<ADIE) | (0<<ADPS2) | (0<<ADPS1) | (1<<ADPS0);
168 SFIOR=(1<<ADHSM) | (0<<ADTS2) | (0<<ADTS1) | (0<<ADTS0);
169
170 // SPI initialization
171 // SPI disabled
172 SPCR=(0<<SPIE) | (0<<SPE) | (0<<DORD) | (0<<MSTR) | (0<<CPOL) | (0<<CPHA) | (0<<SPR1) | (0<<SPR0);
173
174 // TWI initialization
175 // TWI disabled
176 TWCR=(0<<TWIEA) | (0<<TWSTA) | (0<<TWSTO) | (0<<TWEN) | (0<<TWIE);

```

```

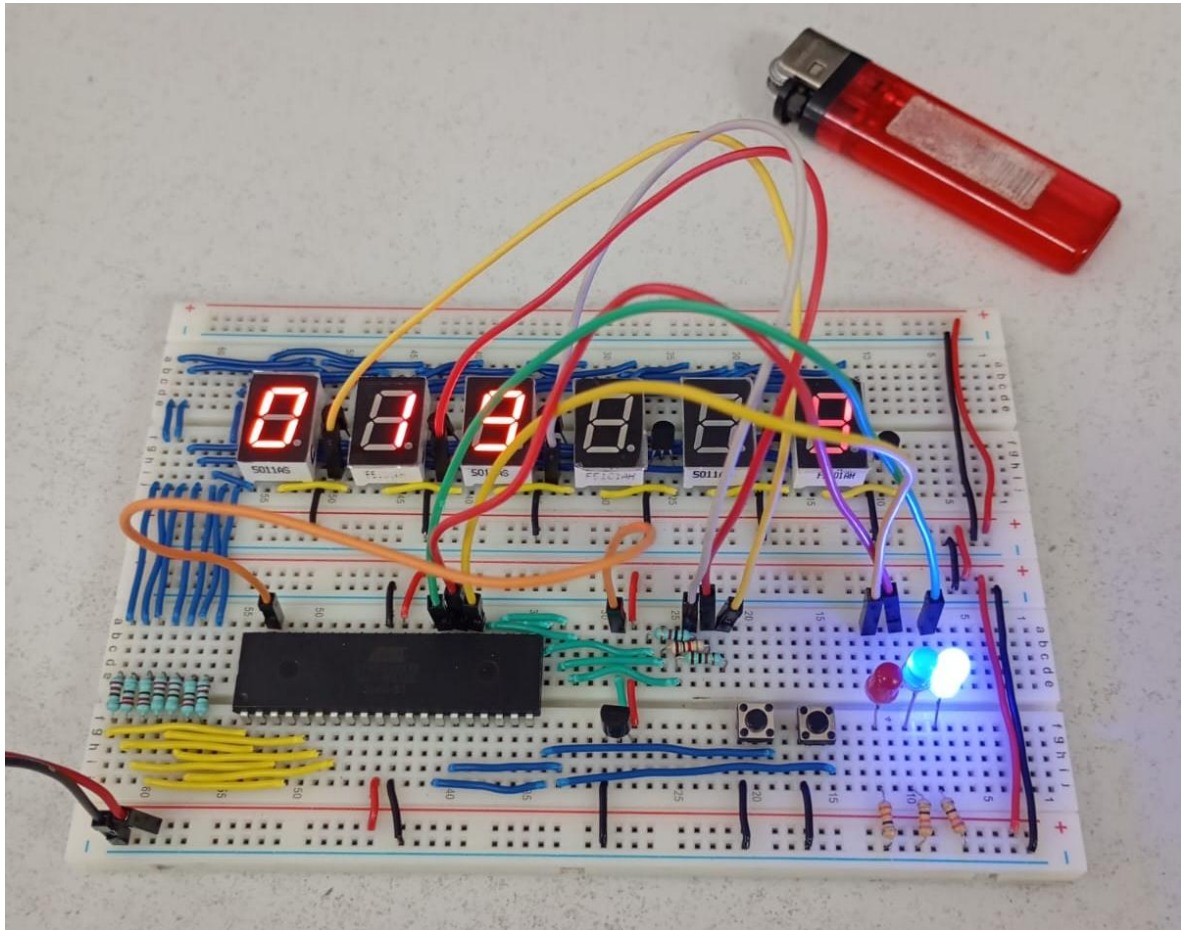
178 while (1) {
179     // Place your code here
180     cn = read_adc(1) * 500 / 255;
181
182     // Manejo pulsador Celsius/Fahrenheit
183     actualCF = BTN_CF;
184     if (pasadoCF && !actualCF) {
185         isFahrenheit = !isFahrenheit;
186         delay_ms(20);
187     }
188     pasadoCF = actualCF;
189
190     // Manejo pulsador de modo
191     actualMODE = BTN_MODE;
192     if (pasadoMODE && !actualMODE) {
193         modo = (modo + 1) % 3;
194         max_temp = 0; // Resetear m̃ximos
195         min_temp = 0xFFFF; // Resetear m̃nimos
196         delay_ms(20);
197     }
198     pasadoMODE = actualMODE;

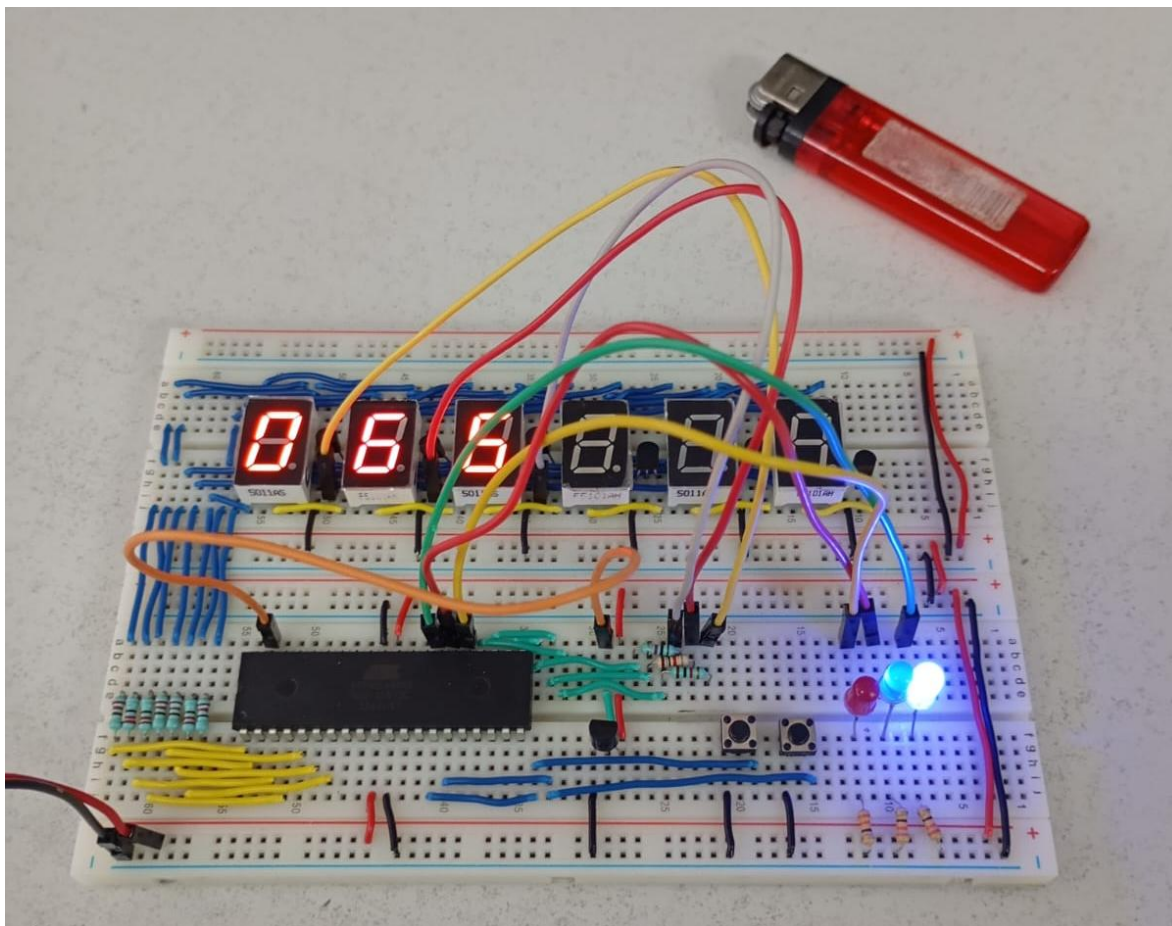
```

```
200 // Determinar valor a mostrar segun modo
201 switch(modo) {
202     case 0: // Tiempo real
203         intensidad = isFahrenheit ? (9 * cn / 5 + 32) : cn;
204         LED_BLUE = 1;
205         LED_GREEN = 0;
206         LED_RED = 0;
207         break;
208
209     case 1: // M3xima temperatura
210         if(cn > max_temp) max_temp = cn;
211         intensidad = isFahrenheit ? (9 * max_temp / 5 + 32) : max_temp;
212         LED_BLUE = 0;
213         LED_GREEN = 1;
214         LED_RED = 0;
215         break;
216
217     case 2: // M3nima temperatura
218         if(cn < min_temp) min_temp = cn;
219         intensidad = isFahrenheit ? (9 * min_temp / 5 + 32) : min_temp;
220         LED_BLUE = 0;
221         LED_GREEN = 0;
222         LED_RED = 1;
223         break;
224 }
```

```
226 // Convertir intensidad a d3gitos
227 centenas = intensidad / 100;
228 decenas = (intensidad % 100) / 10;
229 unidades = intensidad % 10;
230
231 // Multiplexado display
232 PORTB = tabla7segmentos[centenas];
233 C0 = 0; C1 = 0; C2 = 1;
234 delay_ms(5);
235
236 PORTB = tabla7segmentos[decenas];
237 C0 = 0; C1 = 1; C2 = 0;
238 delay_ms(5);
239
240 PORTB = tabla7segmentos[unidades];
241 C0 = 1; C1 = 0; C2 = 0;
242 delay_ms(5);
243 }
244 }
```

Circuito f3isico





CONCLUSIÓN

- **García Escamilla Bryan Alexis**

Anteriormente ya se había usado el ADC en la práctica 15 para interpretar la salida de un potenciómetro y para ello se usaron $(50 * \text{valor}) / 255$ y $(300 * \text{valor}) / 255$ para 0-5 V y 0-30 V respectivamente, sin embargo, en esta práctica esta operación cambia puesto que ya tenemos una interpretación previa y es que 10 mV en la salida del LM35 representa 1 °C y para aprovechar esta relación lo que se hace es `read_adc(1) * 500 / 255` para convertir el valor leído a los milivolts exactos que corresponden a los °C y al final lo que se hace es obtener las centenas, decenas y unidades de este valor y mostrarlos en los displays.

Para mostrar la temperatura máxima y mínima registrada, lo que se hace es comparar el valor actual leído con las últimas temperaturas registradas en ambos casos y registrar las nuevas temperaturas. Cabe destacar que estas actualizaciones no se realizan constantemente sino solo cuando se selecciona el modo de visualización de la temperatura.

Por último, para la conversión de °C a °F se aplica la fórmula $(9 * cn / 5 + 32)$, en donde cn son los °C en milivolts. La fórmula se aplica también para la temperatura máxima y mínima tomándolas como referencia.

- **Meléndez Macedonio Rodrigo**

Para esta práctica nuevamente se vuelve a utilizar el convertidor analógico digital (ADC) realizado para una de las practicas anteriores, sin embargo, aquí su funcionamiento cambia un poco pues debemos tomar en cuenta que 10 mV equivalen a 1 °C dentro del LM35, por lo tanto, se realizó una pequeña conversión para que el valor fuera exacto y correspondiente a lo que se requería para esta práctica.

Esta práctica presenta 3 botones principales con los cuales se puede mostrar la temperatura, el primero es para mostrar la temperatura mínima, el segundo para mostrar la temperatura actual y el tercero para mostrar la temperatura máxima. Cuando queremos mostrar la temperatura máxima y mínima hay un par de detalles a considerar, pues para registrar tanto la temperatura máxima y mínima se tiene que hacer inmediatamente cuando el valor de la temperatura actual está en su máximo en su minimo, de lo contrario si vemos que la temperatura actual va decrementando, el valor que este en ese momento es el que quedará guardado. Por último, esta práctica también tiene presente la conversión de grados Celcius (°C) a grados Farenheit (°F), lo cual no es algo muy complejo pues simplemente se aplica la formula estándar para convertir de grados Celcius a grados Farenheit y el cambio se e reflejado inmediatamente en cualquiera de los 3 casos anteriormente mencionados.

BIBLIOGRAFÍA

(s.f.). *Temperature Sensors*. Omega Engineering. Recuperado el 17 de marzo de 2026. <https://www.dwyeromega.com/en-us/resources/temperature-sensors>

(s.f.). *LM35 Datasheet*. Alldatasheet. Recuperado el 17 de marzo de 2026. https://www.alldatasheet.com/view.jsp?Searchword=lm35&gad_source=1&gad_campaignid=156181564&gbraid=0AAAAADcdDU_aTPbKKoCcWjyy-SssKBbtJ&gclid=Cj0KCQjw9-PNBhDfARIsABHN6-0hCy7Xgrqk14tlC51vbbFtlxJcU3fc2iaAsL-HSGfjn9Ysvu7RF78aAm69EALw_wcB